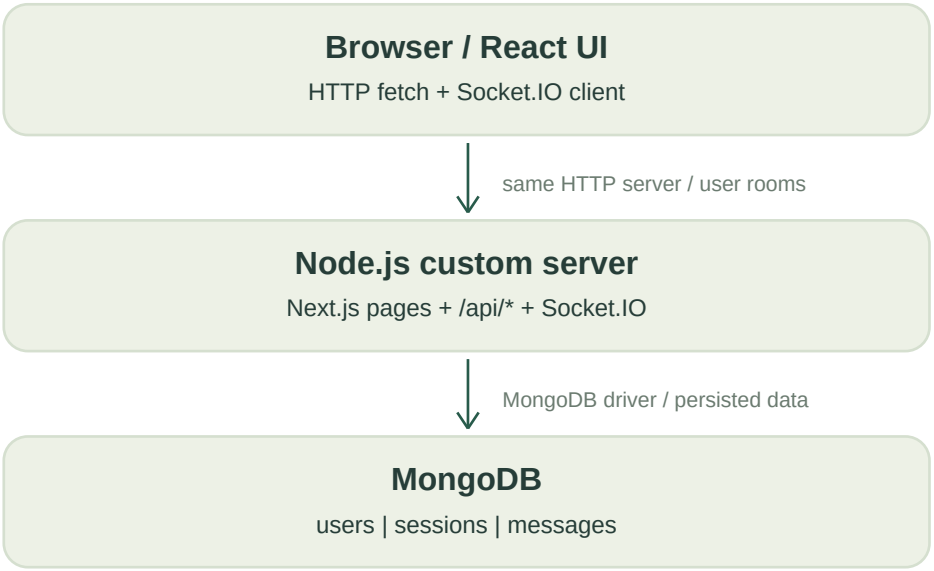
Hello Chat: সম্পূর্ণ প্রজেক্ট আর্কিটেকচার

বাংলায় প্রযুক্তিগত নির্দেশিকা

বর্তমান বাস্তবায়নের ভিত্তিতে • ২০ অক্টোবর ২০২৬

এই অ্যাপ দিয়ে ব্যবহারকারী অ্যাকাউন্ট খুলে অন্য ব্যবহারকারীর সঙ্গে ব্যক্তিগতভাবে চ্যাট করতে পারেন। মেসেজ সংরক্ষণ, Bold ও Italic লেখা, Delete for everyone এবং চার ধরনের emoji reaction রয়েছে।



মূল নকশা

এটি একটি single-server অ্যাপ। একই Node.js HTTP server ওয়েব পৃষ্ঠা, REST API এবং Socket.IO connection পরিচালনা করে। MongoDB স্থায়ী তথ্য রাখে; React browser-এর দৃশ্যমান অবস্থা সামলায়।

গুরুত্বপূর্ণ: /api/* endpoint-গুলো server.mjs-এ লেখা। এগুলো Next.js app/api route handler নয়। Next.js App Router মূলত পৃষ্ঠা ও layout তৈরি করছে।

পাঠক্রম: ২. ফোল্ডার ও দায়িত্ব • ৩. Frontend • ৪. Login/session • ৫. Database • ৬. HTTP API • ৭. Real-time flow • ৮. বিশেষ ফিচার • ৯. নিরাপত্তা ও সীমাবদ্ধতা • ১০. চালানো ও পরীক্ষা

ফোল্ডার কাঠামো ও প্রতিটি অংশের দায়িত্ব

```
chat-app/
  app/
    layout.js
    page.js
    globals.css
  lib/
    security.mjs
    formatting.mjs
  test/
    security.test.mjs
    formatting.test.mjs
  server.mjs
  next.config.mjs
  package.json
  package-lock.json
  .env.example
  .env.local
  .gitignore
  README.md
```

ফাইল / অংশ	কাজ
server.mjs	Server boot, MongoDB connection, API, session, rate limit এবং socket events।
app/page.js	Login/signup screen, people list, chat history, composer, delete ও reaction UI।
app/layout.js	HTML layout, ভাষার attribute, title/description এবং global CSS import।
app/globals.css	রং, typography, message bubble, mobile layout ও toolbar।
lib/security.mjs	Password hash/verify, cookie token পড়া এবং message validation।
lib/formatting.mjs	Selection-এ marker যোগ/সরানো; marker থেকে নিরাপদ React node তৈরি।
next.config.mjs	Next.js config; poweredByHeader বন্ধ এবং Turbopack root নির্ধারণ।

node_modules/ ইনস্টল করা dependency রাখে; .next/ হলো generated build/dev output। এগুলো source code নয় এবং .gitignore-এ বাদ দেওয়া আছে। .env.local-ও version control-এ রাখা হয় না।

Frontend: screen, state ও lifecycle

একটি প্রধান client component

app/page.js-এর Home component-এ “use client” আছে। একই component প্রথমে session যাচাই করে, তারপর authentication screen অথবা chat screen দেখায়। React state দিয়ে UI বদলায়; আলাদা state-management library নেই।

State / ref	ব্যবহার
user, loading, signup	Session result, initial loading এবং login/signup mode।
users, peer, search	সর্বোচ্চ ২০০ জনের তালিকা, নির্বাচিত ব্যক্তি ও client-side search।
messages, draft	বর্তমান conversation ও composer-এর লেখা।
busy, connected, error	পাঠানোর অবস্থা, socket connection এবং ব্যবহারকারীকে error দেখানো।
historyLoading, deleting, reacting	History request, deletion ও reaction request চলার অবস্থা।
socket, active, end	Socket instance, সক্রিয় peer ID এবং scroll-এর শেষ অবস্থান।
deletedIds, reactionUpdates	History ও live event একসঙ্গে এলে পুরোনো তথ্য ফিরে আসা ঠেকানোর cache।
composerInput	Text selection, formatting ও input focus।

পৃষ্ঠা খোলার পর কী হয়

১. GET /api/me দিয়ে session পড়া হয়। ২. Login থাকলে users list লোড হয় এবং socket খোলে। ৩. Users list প্রতি ১৫ সেকেন্ডে refresh হয়। ৪. Peer নির্বাচন করলে সর্বশেষ ১০০ মেসেজ আনা হয়। ৫. Live event অনুযায়ী messages state বদলায় এবং নিচে scroll হয়।

Peer বদলালে পুরোনো history request-এর ফল ignore করা হয়। Message ID দিয়ে duplicate বাদ দেওয়া হয়। Component cleanup-এ timer বন্ধ এবং socket disconnect করা হয়। Mobile-এ sidebar ও conversation পর্যায়ক্রমে দেখা যায়।

Authentication: signup থেকে logout

Signup ও login

POST /api/signup name, email ও password গ্রহণ করে। Email trim করে lowercase করা হয়। নাম ১-৬০ অক্ষর, password ৮-১২৮ অক্ষর এবং email সর্বোচ্চ ২৫৪ অক্ষর হতে পারে। Email unique index duplicate account আটকায়।

Password-এর সঙ্গে ১৬-byte random salt ব্যবহার করে scrypt hash তৈরি হয়। Database-এ salt:hash আকারে রাখা হয়; raw password রাখা হয় না। Login-এ নতুন hash গণনা করে timingSafeEqual দিয়ে মিল দেখা হয়।

Session তৈরি ও যাচাই

সফল signup/login-এ ৩২-byte random token তৈরি হয়। Browser-এ session cookie-তে token যায়; MongoDB sessions collection-এ সেই token-এর SHA-256 digest রাখা হয়। Session-এর মেয়াদ ৭ দিন।

```
Browser: session=<random token>
Cookie: HttpOnly; SameSite=Lax; Path=/; Max-Age=604800
Production: Secure
Database session _id: SHA-256(token)
```

authenticate() cookie থেকে token নেয়, digest দিয়ে session খোঁজে, expiresAt বর্তমান সময়ের পরে কি না দেখে এবং সংশ্লিষ্ট user লোড করে। API ও socket handshake এই function ব্যবহার করে। Socket-এর প্রতিটি send/delete/react event-এও session আবার যাচাই হয়।

Logout-এর আচরণ

POST /api/logout বর্তমান token-এর session মুছে দেয় এবং cookie expire করে। একই user room-এর সব socket disconnect করে। তবে অন্য browser-এর আলাদা session record মুছে যায় না; সেই browser পুনরায় সংযোগ করতে পারে।

সফল response-এ publicUser শুধু id, name ও email দেয়। Password hash frontend-এ পাঠানো হয় না।

MongoDB: collection, সম্পর্ক ও index

MongoDB Node.js driver সরাসরি ব্যবহার হয়েছে; Mongoose নেই। db() প্রথম ব্যবহারে connection ও index তৈরি করে promise cache করে। Connection ব্যর্থ হলে cache reset হয়, যাতে পরে আবার চেষ্টা করা যায়।

Collection	প্রধান field
users	_id: ObjectId; name: string; email: string; password: salt:hash; createdAt: Date
sessions	_id: SHA-256 token string; userId: string; expiresAt: Date
messages	_id: ObjectId; from/to: user ID string; text: string; conversation: string; createdAt: Date; deleted?: boolean; reactions?: object; reactionVersion?: number

সম্পর্ক

একজন user-এর একাধিক session এবং অনেক message থাকতে পারে। Message-এর from ও to users._id-এর string form ধরে রাখে। Database foreign-key constraint নেই; অ্যাপ sender ও recipient যাচাই করে।

```
conversation = [senderId, recipientId].sort().join(":")
reactions = { "<userId>": "<allowed emoji>" }
```

দুই ID sort করে একই conversation key বানানো হয়। ফলে A□B ও B□A একই history-তে আসে। reactions map-এ প্রতিজনের একটি reaction থাকে; reactionVersion প্রতিটি reaction update-এ বাড়ে।

Index	উদ্দেশ্য
users: { email: 1 }, unique	একই email দিয়ে একাধিক account বন্ধ।
sessions: { expiresAt: 1 }, TTL=0	মেয়াদ শেষ হলে MongoDB background process session সরায়; authentication-এও expiry দেখা হয়।
messages: { conversation: 1, createdAt: -1 }	Conversation অনুযায়ী সর্বশেষ মেসেজ query দ্রুত করা।

Delete for everyone-এ document থাকে, কিন্তু text ও reactions field সরানো হয় এবং deleted=true হয়। Metadata ও tombstone থাকে। এটি backup বা আগে নেওয়া screenshot মুছে দেওয়ার ব্যবস্থা নয়।

HTTP API: request ও response contract

Browser-এর api() helper fetch করে এবং JSON parse করে। Response সফল না হলে error message throw করে। API response-এ Cache-Control: no-store দেওয়া হয়।

Method / endpoint	Input ও result
POST /api/signup	{ name, email, password } → { user }; session cookie সেট হয়।
POST /api/login	{ email, password } → { user }; নতুন session cookie।
GET /api/me	Cookie → { user }; login না থাকলে 401।
POST /api/logout	Cookie → { ok: true }; session/cookie সরায়, socket disconnect।
GET /api/users	Cookie → { users }; নিজেকে বাদ দিয়ে নাম অনুযায়ী সর্বোচ্চ ২০০ জন।
GET /api/messages?with=<peerId>	Cookie + peer ID → { messages }; সর্বশেষ ১০০টি মেসেজ chronological order-এ।

History access কীভাবে সীমিত হয়

Server নিজের authenticated user ID এবং query-র peer ID থেকে conversation key বানায়। Client অন্য কারও sender ID দিয়ে নিজের পরিচয় বদলাতে পারে না। ObjectId format যাচাই করা হয়।

Error ও সীমা

ভুল input সাধারণত 400, ভুল login বা missing session 401, invalid origin 403, duplicate signup 409 এবং অজানা API 404 দেয়। Generic request/database failure এবং কিছু rate-limit error বর্তমানে 503 হয়; আলাদা 429 response নেই।

Request body-এর সীমা ১৬,০০০ byte। User list-এ pagination নেই এবং history-তে “load older messages” নেই। Registered user-এর email বর্তমানে অন্য logged-in user-এর তালিকায় দেখা যায়।

Socket.IO: real-time message lifecycle

একই HTTP server-এ Socket.IO attach করা আছে। Handshake-এ session validate হয়। সফল connection-এ socket নিজের user ID নামের room-এ যোগ দেয়। এক user-এর একাধিক tab একই room-এর event পায়।

Event / direction	Payload ও কাজ
message:send | client server	{ to, text }; session, rate, recipient ও text যাচাই; MongoDB insert।
message:new | server clients	সংরক্ষিত message + id; sender ও recipient room-এ broadcast।
message:delete | client server	{ id }; কেবল sender document update করতে পারেন।
message:deleted | server clients	{ id, from, to }; দুই room-এর UI-তে deleted placeholder।
message:react | client server	{ id, emoji }; দুই participant-এর যে কেউ নিজের reaction বদলাতে পারেন; null মানে সরানো।
message:reactions | server clients	{ id, reactions, reactionVersion }; দুই room-এর reaction state update।

মেসেজ পাঠানোর ধাপ

১. Composer submit হলে draft পাঠানো হয়। ২. Server session থেকে sender নির্ধারণ করে। ৩. Text trim করে ১-২০০০ অক্ষর নিশ্চিত করে এবং recipient খোঁজে। ৪. MongoDB insert সফল হলে event broadcast হয়। ৫. Ack-এ ok এলে composer খালি হয়।

Client acknowledgement-এর জন্য ১০ সেকেন্ড অপেক্ষা করে। Timeout মানেই message database-এ যায়নি, এমন নয়। Client-generated message ID বা retry deduplication নেই; না দেখে retry করলে duplicate হতে পারে।

Socket.IO reconnect করে, কিন্তু disconnect-এর সময়ে মিস হওয়া event স্বয়ংক্রিয়ভাবে history থেকে sync করার ব্যবস্থা নেই। Conversation আবার খুললে history নতুন করে পাওয়া যায়।

Formatting, deletion ও emoji reaction

Bold ও Italic

Composer-এর B/I button অথবা Ctrl/Command+B ও Ctrl/Command+I selection-এর চারপাশে marker বসায়। একই selection-এ আবার ব্যবহার করলে marker সরতে পারে। Selection খালি হলে “text” placeholder বসে; পাঠানোর আগে preview দেখা যায়।

```
**bold**  -> strong
*italic*  -> em
***both*** -> strong + em
```

MongoDB-তে marker-সহ মূল string থাকে। formatMessage() browser-এ strong/em React element তৈরি করে। Raw HTML execute করে না। এটি সীমিত formatting parser; পূর্ণ Markdown, arbitrary nesting বা rich-text editor নয়।

Delete for everyone

Button শুধু নিজের পাঠানো, এখনও মুছে না যাওয়া মেসেজে থাকে। Server query-তে message ID-এর সঙ্গে from=user.id মেলায়। Update-এ deleted=true হয় এবং text/reactions unset হয়। দুই পক্ষ “This message was deleted” দেখেন। একই deletion retry করা যায়।

Love, Like, Angry ও Happy

অনুমোদিত reaction: Love (heart), Like (thumbs up), Angry (angry face), Happy (smiling face)। প্রতিজন একটি reaction দিতে পারেন। অন্য emoji চাপলে সেটি বদলায়; নির্বাচিত emoji আবার চাপলে null পাঠিয়ে সরায়। UI-তে প্রতিটি emoji-এর count দেখায়।

Server user ID নিজে নেয় এবং শুধু reactions.<userId> field update করে। Atomic findOneAndUpdate participant ও deleted=false শর্ত যাচাই করে। তাই অন্য user-এর reaction overwrite বা deleted message-এ reaction দেওয়া যায় না।

reactionVersion পুরোনো event দিয়ে নতুন reaction হারানো ঠেকায়। deletedIds cache পুরোনো history দিয়ে মুছে যাওয়া লেখা ফেরত আসা ঠেকায়। এগুলো browser memory-তে থাকে; MongoDB-ই স্থায়ী source of truth।

নিরাপত্তা, বর্তমান সীমা ও উন্নয়নের জায়গা

ব্যবস্থা	বর্তমান বাস্তবায়ন
Password ও session	Salted scrypt; timing-safe comparison; random token; database-এ token digest।
Cookie	HttpOnly, SameSite=Lax; production-এ Secure; ৭ দিনের expiry।
Origin check	Mutation HTTP request ও socket handshake-এ Origin থাকলে Host মেলে কি না দেখা হয়।
Authorization	Send-এ server-side sender; delete-এ sender-only; reaction-এ participant-only।
Input / rendering	Body/socket payload সীমা; message ২০০০ অক্ষর; emoji allowlist; raw HTML render নয়।
Rate limit / minute	Auth: প্রতি IP-তে ২০; send: প্রতি user-এ ৬০; delete: ৬০; reaction: ১২০।

একটি server instance-এর জন্য তৈরি

Rate-limit Map ও Socket.IO room process memory-তে থাকে। Restart-এ limit reset হয়। একাধিক instance চালাতে shared Socket.IO adapter, প্রয়োজনমতো sticky-session setup এবং distributed rate limiter দরকার।

বর্তমানে যা নেই

End-to-end encryption, email verification, password reset, block/report, attachment, group chat, typing indicator, read receipt ও নির্ভরযোগ্য user online-presence নেই। “Connected” নিজের socket-এর অবস্থা বোঝায়; অন্য user online আছেন কি না বোঝায় না।

Production-এর আগে বিবেচনা

HTTPS ও proxy/WebSocket config, database access control, backup/restore, log/monitoring, pagination, reconnect catch-up এবং idempotent send যোগ করা উচিত। Email visibility-এর নীতি ও account recovery-ও নির্ধারণ করা প্রয়োজন। এগুলো প্রস্তাবিত উন্নয়ন; বর্তমান code-এ যোগ করা নেই।

scryptSync password কাজের সময়ে event loop block করে। বড় traffic-এ async hashing বা worker এবং shared infrastructure বিবেচনা করা যায়।

চালানো, deployment ও যাচাই

Local setup

Node.js 20.9 বা পরের সংস্করণ এবং local MongoDB অথবা Atlas connection লাগবে। নিচের উদাহরণটি সাধারণ setup; নিজের database URL দিন। গোপন connection string PDF-তে রাখা হয়নি।

```
cd outputs/chat-app
npm install
cp .env.example .env.local
# Edit .env.local
npm run dev

MONGODB_URI=mongodb://127.0.0.1:27017
MONGODB_DB=hello_chat
PORT=3000
```

বর্তমান কাজের local database 27018 port-এ চালানো হয়েছিল; সাধারণ .env.example-এ 27017 আছে। নিজের চালু database-এর port অনুযায়ী URL মিলিয়ে নিতে হবে। localhost:3000 খুলে আলাদা browser profile/incognito-তে দুই account দিয়ে পরীক্ষা করা যায়।

Production deployment

```
npm run build
npm start
```

Persistent Node.js process এবং WebSocket সমর্থন করে এমন host দরকার। এই custom server Vercel-এর serverless function হিসেবে সরাসরি deploy করার জন্য তৈরি নয়। Production-এ HTTPS ব্যবহার করুন, কারণ session cookie Secure থাকে।

যা যাচাই করা হয়েছে

Unit test: password hashing/verification, cookie parsing, message validation, formatting render ও selection toggle। Integration script: signup/login, authenticated users, live delivery, saved history, private conversation, logout; sender-only delete; চার reaction, replacement/removal, persistence এবং outsider/invalid input rejection। Production build-ও সফল হয়েছে।

npm test শুধু test/ directory-এর unit tests চালায়। Integration scripts বর্তমান workspace-এর work/ directory-তে আছে; এগুলো package script বা repository-এর স্থায়ী test suite-এ যুক্ত করা হয়নি।

এই নথির ভিত্তি

যাচাই করা source: server.mjs; app/page.js, layout.js, globals.css; lib/security.mjs, formatting.mjs; test/*.test.mjs; package.json, next.config.mjs, README.md এবং work/-এর integration scripts। নথিটি বর্তমান code snapshot ব্যাখ্যা করে; ভবিষ্যৎ পরিবর্তন স্বয়ংক্রিয়ভাবে এখানে যুক্ত হবে না।